DevDocs — RAG Answer Generation

1. Overview

RAG Answer Generation is responsible for generating grounded, developer-friendly answers using:

- The user's current question
- Relevant conversation context
- Fresh project-scoped retrieval results
- System instructions

The LLM must answer using the retrieved project context as the primary source for project-specific information.

The goal is to provide useful answers without allowing the LLM to invent unsupported project behavior.

---

2. Answer Generation Inputs

For every new user question, the LLM receives:

```
System Instructions
        +
Conversation Context
        +
Fresh Retrieved Project Context
        +
Current User Question
        ↓
Answer-Generation LLM
        ↓
Answer
```

The entire project is never sent to the LLM.

The entire conversation history is also never sent directly to the LLM.

Only the relevant conversation context and retrieved project context are included.

---

3. Fresh Retrieval for Every Question

Every new user question triggers a **fresh project-scoped retrieval**.

Previously retrieved chunks are not automatically reused as factual context for subsequent questions.

Example:

```
Question 1:
"How does authentication work?"

→ Fresh retrieval
→ auth.py
→ jwt.py
→ middleware.py


Question 2:
"How are database connections created?"

→ Fresh retrieval
→ database.py
→ config.py
→ connection.py
```

Conversation context can help the LLM understand references such as:

> "Where is the token generated?"

However, project information for the new question must come from a fresh retrieval.

---

4. Context Authority

Different parts of the prompt have different purposes.

System Instructions

Define how the LLM should behave.

Conversation Context

Provides conversational continuity and helps interpret references to previous discussion.

Retrieved Project Context

Acts as the **primary source of truth for project-specific information**.

Current User Question

Defines what the user currently wants to know.

The core rule is:

> **General knowledge can explain what the retrieved code means, but it cannot fill gaps in what the project actually does.**

---

5. Strict Grounding

The LLM must rely on retrieved project context when making project-specific claims.

The LLM must not:

- Guess missing implementation details
- Invent files, functions, classes, APIs, or configurations
- Assume common project patterns are being used
- Use phrases such as "probably", "most likely", or similar speculation for unsupported project behavior

For example, if retrieved code contains:

```
jwt.encode(payload, secret_key, algorithm="HS256")
```

the LLM may explain what `jwt.encode()` generally does.

However, if the retrieved context does not show where `secret_key` comes from, the LLM must not claim:

> "The secret is probably stored in `.env`."

Instead, it should state that the retrieved project context does not show where the secret is obtained.

---

6. No Relevant Context

If the retrieval pipeline determines that there is **no sufficiently relevant project context**, the answer-generation LLM should not be called.

Flow:

```
User Question
      ↓
Fresh RAG Retrieval
      ↓
No Relevant Context
      ↓
Do NOT call Answer LLM
```

```
      ↓
Show fallback message
```

Example fallback:

> I couldn't find relevant information in this project to answer that question.

This prevents unnecessary LLM calls and prevents the LLM from attempting to answer without project evidence.

---

7. Insufficient Context

A different situation occurs when retrieval finds **some relevant context**, but that context is not sufficient to completely answer the question.

In this case, the LLM **is called**.

The LLM should:

1. Answer what can be determined from the retrieved context.
2. Clearly identify what cannot be determined.
3. Never fill the missing information using assumptions.

Example:

Retrieved context shows:

```
jwt.encode(payload, secret_key, algorithm="HS256")
```

but does not show where `secret_key` is obtained.

The answer can say:

> The JWT is signed using `secret_key` in `auth/jwt.py`, but the retrieved project context does not show where `secret_key` is obtained.

---

8. Developer-Friendly Answers

DevDocs is designed for developers understanding an existing project.

Answers should therefore be capable of explaining:

- Files
- Classes

- Functions
- Code flow
- Relationships between components
- API behavior
- Configuration
- Implementation details

The response should be adapted to the user's question rather than following a rigid template.

---

9. Adaptive Markdown

DevDocs responses use **Markdown**, but the LLM decides which Markdown elements are useful.

No fixed response structure is required.

The LLM may use:

- Paragraphs
- Headings
- Bulleted lists
- Numbered steps
- Tables
- Code blocks
- Inline code

depending on the question.

Simple question

A short paragraph may be sufficient.

Flow question

Numbered steps may be appropriate.

Code explanation

The answer may contain explanations followed by code blocks.

The principle is:

> **Use the Markdown structure that best communicates the answer.**

---

10. Adaptive Answer Length

The LLM decides the natural length of the answer.

Simple questions should generally receive concise answers.

Complex questions, code flows, or architectural explanations may require more detail.

There is no fixed answer length such as:

```
Always 200 words
Always 500 words
Always 5 bullet points
```

Instead:

> **The LLM should provide as much detail as necessary to clearly answer the user's question, without unnecessary explanation.**

This decision happens within the normal answer-generation LLM call and does not require an additional LLM call.

---

11. Simplified / Dummy Code

For code-understanding questions, DevDocs may provide simplified or dummy code when it helps the user understand the actual implementation.

Example:

```python
def validate_token(token):
    payload = decode_jwt(token)

    if not payload:
        raise InvalidTokenError()

    return payload
```

Such code must be clearly labelled, for example:

> **Simplified example**

The simplified example must not introduce unsupported project-specific behavior.

The actual project implementation remains authoritative.

If the actual project code is already simple enough, a dummy example is unnecessary.

---

12. Streaming Responses

DevDocs will support **streaming LLM responses**.

Instead of waiting for the complete answer, the backend forwards generated output to the frontend as it becomes available.

```
React
  ↓
FastAPI
  ↓
RAG Retrieval
  ↓
Context Builder
  ↓
Streaming LLM
  ↓
FastAPI
  ↓
React
```

The answer progressively appears in the UI.

Streaming is provider/model agnostic and should not make DevDocs dependent on a particular LLM.

---

13. Thinking State

Before the first streamed token arrives, the frontend displays:

> **Thinking...**

The user does not need to see individual internal processing stages such as:

- Searching project
- Running BM25
- Running vector search
- Reranking
- Building prompt
- Generating answer

The V1 UI uses only:

```
Thinking...
```

Once the first answer token arrives, the `Thinking...` state is replaced by the streamed response.

14. Stop Generation

The user can stop an active streaming generation.

Flow:

```
LLM Streaming
      ↓
User clicks Stop
      ↓
Cancel active generation
      ↓
Keep partial response temporarily visible
      ↓
Generation ends
```

The user can then send another question after the cancellation is completed.

The stopped response is treated as an **incomplete message**.

It is not saved to conversation history.

---

15. Temporary Partial Responses

Partial responses exist only temporarily while generation is in progress or while an incomplete generation is being displayed.

They are not persisted as normal assistant messages.

Example:

```
LLM
 ↓
"The authentication flow starts..."
 ↓
"The authentication flow starts in the middleware..."
 ↓
Generation interrupted
```

The partial response may remain visible temporarily, but it is not saved to the messages database.

If the frontend is refreshed, the incomplete response disappears because it was never persisted.

Navigation away from the conversation does not guarantee that an incomplete response will survive.

16. Successful Generation Persistence

An assistant message is saved to the database **only after successful generation**.

Example:

```
User message
     ↓
RAG
     ↓
LLM streaming
     ↓
Complete answer
     ↓
Save assistant message
```

The database therefore contains completed assistant responses rather than incomplete generations.

---

17. User Message Persistence

The user's message is saved when it is submitted.

This is independent of whether answer generation succeeds.

Therefore:

```
User question → saved immediately
Assistant answer → saved only after successful generation
```

If generation fails, the user's question remains in the conversation history.

---

18. LLM/API Failure

If the LLM/API fails before or during generation, DevDocs uses a limited automatic retry mechanism.

Conceptually:

```
LLM Request
    ↓
Failure
    ↓
```

```
Retry
    ↓
Failure
    ↓
Retry
    ↓
All retries exhausted
    ↓
Show error
```

The exact retry count is a configuration/implementation decision.

---

19. Failure Before Streaming

If the LLM fails before any response has been generated:

```
User Question
      ↓
LLM Request
      ↓
Failure
      ↓
Automatic Retry
      ↓
Success → Stream Answer

OR

All Retries Fail
      ↓
Show Error
```

No assistant message is saved if generation ultimately fails.

The user's original message remains saved.

---

20. Failure During Streaming

If an LLM/API failure occurs after streaming has already started:

```
LLM
 ↓
Partial Answer
```

```
 ↓
API Failure
```

The partial response remains temporarily visible.

It is marked as incomplete/interrupted.

The user is given a **Retry** action.

The incomplete response is not saved as an assistant message.

---

21. Retry After Streaming Failure

When the user clicks **Retry**:

```
Partial Answer
      ↓
User clicks Retry
      ↓
Remove partial answer
      ↓
Run RAG again
      ↓
Run LLM again
      ↓
Stream new answer
```

The failed attempt is treated as if it never became a completed assistant response.

The retry starts a fresh generation.

---

22. Retrieval/System Failure

If the RAG/retrieval pipeline itself fails due to a technical problem, such as:

- Vector database failure
- BM25 retrieval failure
- Embedding service failure
- Reranker failure
- Other retrieval infrastructure errors

DevDocs should:

1. Retry a limited number of times.
2. If retries succeed, continue normally.

3. If all retries fail, show an error to the user.
4. Do not generate an answer without successful retrieval.

The user's message remains saved.

No assistant answer is generated or saved.

---

23. Output Token Limit

The LLM has a maximum output-token limit as a **technical safety ceiling**, not as the desired answer length.

The LLM still decides the natural answer length.

Conceptually:

```
LLM determines natural answer length
              ↓
        Technical ceiling
              ↓
       Complete answer
```

The ceiling should be sufficiently generous so normal DevDocs answers do not reach it.

The exact value depends on the selected LLM and will be configured later.

---

24. Output Limit Truncation

If generation reaches the maximum output-token limit, the answer may technically be truncated.

For V1:

- The limit should be set generously.
- Automatic continuation using another LLM call is **not implemented**.
- Automatic continuation is considered a future enhancement.
- DevDocs should not intentionally use the token limit as a way of controlling normal answer length.

The normal expectation is that the LLM completes its answer naturally before reaching the technical ceiling.

---

25. Generation Timeout

DevDocs should protect against an LLM request becoming stuck indefinitely.

There should be a timeout/inactivity mechanism for generation.

Conceptually:

```
LLM streaming
      ↓
Chunks continue arriving
      ↓
Normal generation
```

If generation becomes inactive for too long:

```
No output for too long
      ↓
Cancel generation
      ↓
Handle as interrupted/failed generation
```

The exact timeout values will be configured later based on the selected LLM/provider.

---

26. Conversation Context

The answer-generation LLM receives relevant conversation context rather than the entire conversation history.

The conversation system maintains:

- One stored conversation summary
- Summary checkpoint
- Recent messages after the checkpoint

Conceptually:

```
Stored Summary
      +
Recent Messages
      +
Current Question
      +
Fresh Retrieved Project Context
```

This allows DevDocs to maintain conversational continuity without sending the entire conversation to the LLM.

---

27. Summary and Context Limits

Conversation summarization is triggered based on context size approaching a configured threshold.

The threshold leaves room for:

- System instructions
- Conversation context
- Retrieved project context
- Current question
- LLM output

The exact threshold depends on the selected LLM.

A running token estimate should be maintained rather than recalculating the entire conversation token count on every message.

---

28. Complete V1 Generation Flow

The complete DevDocs answer-generation flow is:

```
User submits question
        ↓
Save user message
        ↓
Display "Thinking..."
        ↓
Fresh project-scoped retrieval
        ↓
Is relevant context available?
        │
        ├── No
        │    ↓
        │  Do not call Answer LLM
        │    ↓
        │  Show fallback message
        │
        └── Yes
             ↓
       Build context
             ↓
       System Instructions
       +
```

```
        Conversation Context
        +
        Retrieved Project Context
        +
        Current Question
              ↓
        Answer LLM
              ↓
        Stream response
              ↓
        Remove "Thinking..."
              ↓
        Display answer progressively
              ↓
        ┌─────────────────────────────┐
        │                             │
    Success                       Failure
        │                             │
        ↓                             ↓
Save complete answer          Retry automatically
        │                             │
        ↓                        ┌────┴────┐
Completed message          Success      Failure
                              │             │
                              ↓             ↓
                         Continue      Show error
                         streaming
```

For a user-initiated stop:

```
Streaming answer
      ↓
User clicks Stop
      ↓
Cancel generation
      ↓
Keep partial answer temporarily visible
      ↓
Do NOT save assistant message
      ↓
User can send another question
```

For a streaming failure:

```
Streaming answer
      ↓
API/LLM failure
      ↓
Partial answer remains temporarily visible
```

```
      ↓
Retry
      ↓
Remove partial answer
      ↓
Fresh RAG + LLM generation
```

---

29. Core V1 Principles

The RAG Answer Generation system follows these principles:

1. **Project context is the source of truth for project-specific claims.**
2. **Fresh retrieval is performed for every user question.**
3. **The LLM must not guess missing project information.**
4. **No relevant context means no answer-generation LLM call.**
5. **Relevant but insufficient context can still be passed to the LLM, with strict uncertainty handling.**
6. **Answers use adaptive Markdown.**
7. **Answer length is adaptive and decided by the LLM.**
8. **LLM responses are streamed to the frontend.**
9. **"Thinking..." is shown before the first streamed token.**
10. **Only successfully completed assistant responses are persisted.**
11. **User messages are saved even when answer generation fails.**
12. **Failed or manually stopped partial responses are temporary and not persisted.**
13. **Retry removes the failed partial response and starts a fresh generation.**
14. **LLM and retrieval failures use limited automatic retries.**
15. **Output-token limits are safety ceilings, not desired answer lengths.**
16. **Automatic continuation after reaching the output limit is a future enhancement.**
17. **Generation timeout/inactivity protection prevents requests from hanging indefinitely.**

---

30. Future Enhancements

The following are intentionally outside the V1 implementation:

- Automatic continuation when the output-token limit is reached
- More sophisticated generation recovery
- Persistent generation records for advanced debugging/recovery
- Advanced streaming analytics
- More sophisticated cancellation/recovery handling
- Provider-specific generation optimizations